Experiment 22: Independent Component Analysis and Component Visualization

Question

Problem Statement: In a hospital ECG recording, the mother's heartbeat masks the foetal heartbeat. Doctors need the two signals separated (the cocktail-party problem). Three synthetic source signals (sine, square and sawtooth, or maternal and foetal ECG) mixed through a random mixing matrix. The objective is to separate mixed signals into independent sources with ICA. Use Scikit-learn, NumPy, Matplotlib.

Tasks:

1. Generate the source signals and mix them.
2. Apply FastICA to recover the sources.
3. Apply PCA to the same mixtures for comparison.
4. Plot original, mixed, ICA-recovered and PCA-recovered signals.
5. Measure recovery quality with correlation to the true sources.

Code

```
"""Experiment 22: Independent Component Analysis and Component Visualization (cocktail-party problem)"""
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from sklearn.decomposition import FastICA, PCA

np.random.seed(0)

# ---------------- Task 1: sources and mixing ----------------
n = 2000
time = np.linspace(0, 8, n)
s1 = np.sin(2 * time)                                  # sine
s2 = np.sign(np.sin(3 * time))                         # square wave
s3 = signal.sawtooth(2 * np.pi * time)                 # sawtooth
S = np.c_[s1, s2, s3]
S += 0.05 * np.random.normal(size=S.shape)             # small measurement noise
S /= S.std(axis=0)                                     # standardise the sources

A = np.random.rand(3, 3)                               # random mixing matrix
X = S @ A.T                                            # observed mixtures (3 "microphones")
print("Mixing matrix A:\n", np.round(A, 3))

# ---------------- Task 2: FastICA ----------------
ica = FastICA(n_components=3, whiten="unit-variance", random_state=0, max_iter=1000)
S_ica = ica.fit_transform(X)

# ---------------- Task 3: PCA on the same mixtures ----------------
pca = PCA(n_components=3)
S_pca = pca.fit_transform(X)

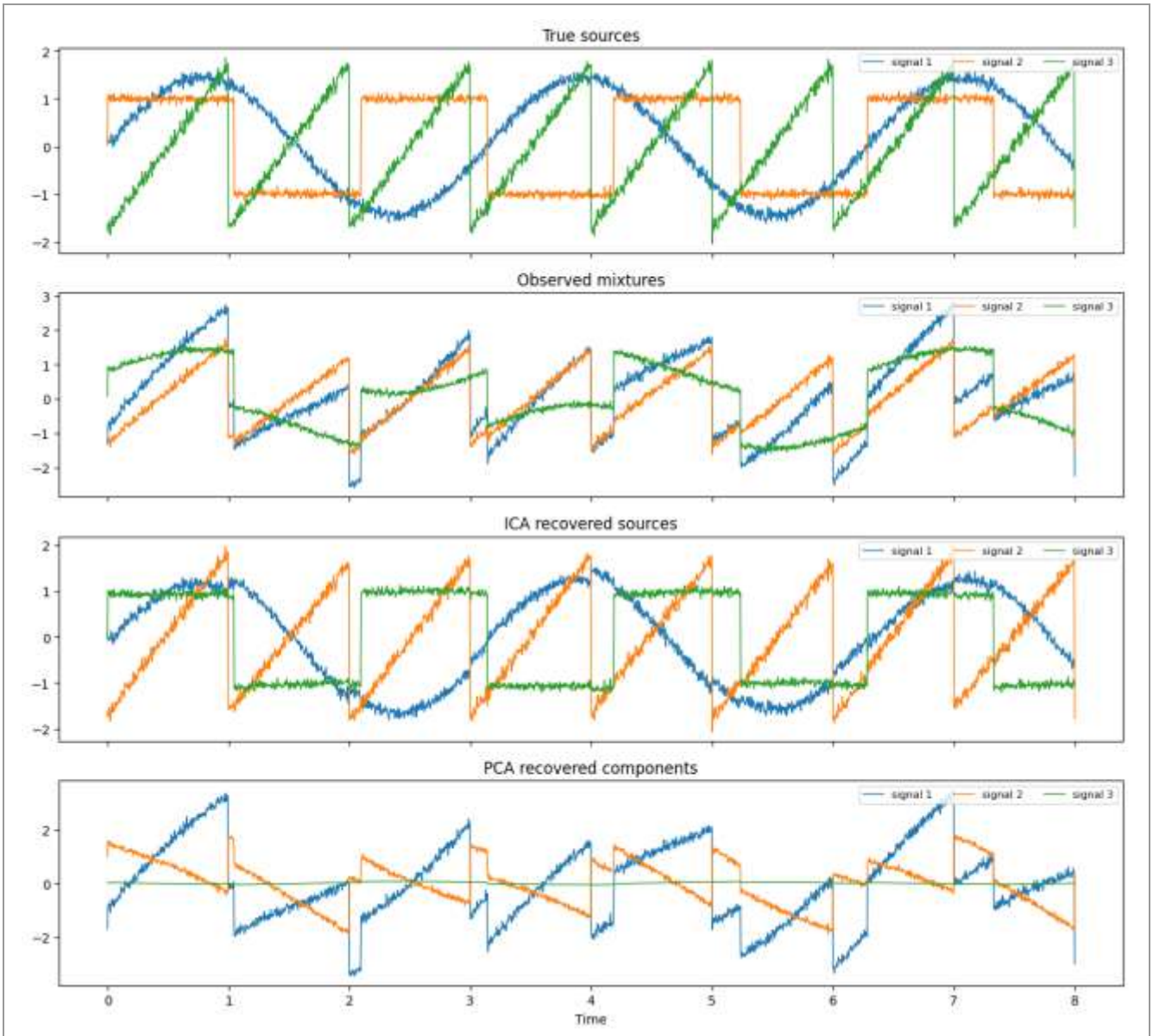
# ---------------- Task 4: plots ----------------
names = ["Sine", "Square", "Sawtooth"]
fig, axes = plt.subplots(4, 1, figsize=(12, 11), sharex=True)
for data, ax, title in zip([S, X, S_ica, S_pca], axes,
                           ["True sources", "Observed mixtures", "ICA recovered sources", "PCA recovered
components"]):
    for i in range(3):
        ax.plot(time, data[:, i], lw=1, label=f"signal {i+1}")
    ax.set_title(title); ax.legend(loc="upper right", ncol=3, fontsize=8)
axes[-1].set_xlabel("Time")
plt.tight_layout(); plt.show()

# ---------------- Task 5: recovery quality ----------------
def match_quality(S_true, S_est):
    C = np.abs(np.corrcoef(S_true.T, S_est.T)[:3, 3:])        # |corr|: ICA/PCA ignore sign and order
    return C, C.max(axis=1)
```

```
C_ica, best_ica = match_quality(S, S_ica)
C_pca, best_pca = match_quality(S, S_pca)
print("\n|Correlation| of true sources (rows) with ICA components (columns):\n", np.round(C_ica, 3))
print("|Correlation| of true sources (rows) with PCA components (columns):\n", np.round(C_pca, 3))
print(f"\n{'Source':10}{'ICA best |corr|':>18}{'PCA best |corr|':>18}")
for nm, a, b in zip(names, best_ica, best_pca):
    print(f"{nm:10}{a:18.4f}{b:18.4f}")
print(f"{'Mean':10}{best_ica.mean():18.4f}{best_pca.mean():18.4f}")

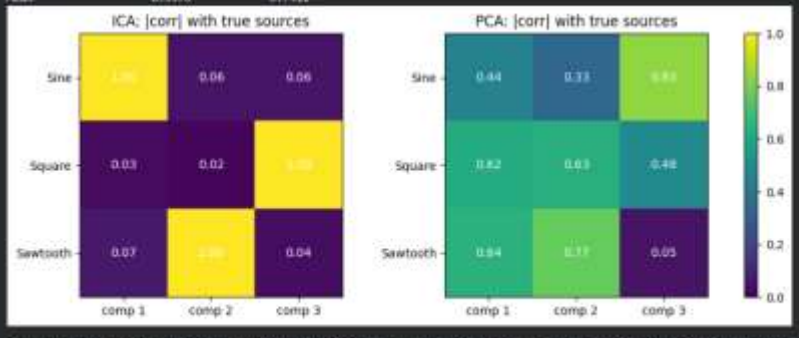
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
for a, C, t in zip(ax, [C_ica, C_pca], ["ICA", "PCA"]):
    im = a.imshow(C, vmin=0, vmax=1, cmap="viridis")
    a.set_xticks(range(3)); a.set_xticklabels([f"comp {i+1}" for i in range(3)])
    a.set_yticks(range(3)); a.set_yticklabels(names); a.set_title(f"{t}: |corr| with true sources")
    for r in range(3):
        for c in range(3):
            a.text(c, r, f"{C[r, c]:.2f}", ha="center", va="center", color="w")
plt.colorbar(im, ax=ax, fraction=0.03); plt.show()
print("\nConclusion: ICA exploits non-Gaussianity/independence and recovers each source almost perfectly; "
      "PCA only finds uncorrelated directions, so its components remain mixtures.")
```

Output




```
[Correlation] of true sources (rows) with ICA components (columns):
 [[0.997 0.056 0.058]
 [0.029 0.017 0.999]
 [0.071 0.997 0.039]]
[Correlation] of true sources (rows) with PCA components (columns):
 [[0.442 0.334 0.832]
 [0.618 0.626 0.478]
 [0.641 0.705 0.054]]

Source       ICA Best |corr|   PCA Best |corr|
Sine                  0.9967            0.8321
Square                0.9994            0.6258
Sawtooth              0.9968            0.7055
Mean                  0.9976            0.7411
```




```
Conclusion: ICA exploits non-Gaussianity/independence and recovers each source almost perfectly; PCA only finds uncorrelated directions, so its components remain mixtures.
```